Evidence-Governed Agentic Reverse Engineering

Behavioral Evidence, Claim Calibration, and an Empirical Research Program for Autonomous Codebase Investigation

Working research synthesis - October 2026

Central proposition

Autonomous reverse engineering can be evaluated as a claim-calibration problem: the quality of an investigation depends on whether behavioral conclusions remain traceable to reproducible evidence with explicit scope, environmental reality, oracles, sensitivity, provenance, and unresolved uncertainty.

Abstract

Software reverse engineering already provides mature techniques for recovering structure, locating features, observing executions, inferring specifications, generating tests, reconstructing architecture, and assessing change risk. Autonomous coding agents add a distinct reliability problem: one system can infer an expected behavior, generate tests and mocks that embody that inference, observe agreement, and report the resulting consistency as support for a behavioral conclusion. The speed and autonomy of agentic investigation increase the value of explicit controls on how evidence becomes claims.

This paper develops evidence-governed agentic reverse engineering as a research proposal within the established fields of reverse engineering and program comprehension. The proposal organizes heterogeneous evidence around six dimensions: proposition, exercised domain, included reality, oracle, sensitivity, and provenance. It also preserves four epistemic categories that are frequently collapsed during code investigation: mechanism, actuality, intent, and correctness. Characterization tests, dynamic traces, software reconnaissance, feature location, property-based testing, differential testing, metamorphic testing, combinatorial testing, fuzzing, symbolic and concolic execution, mutation testing, static analysis, formal methods, repository history, and real-boundary execution become evidence-generation methods whose conclusions depend on the scope of the experiment.

The agentic framing leads to a concrete empirical program. The primary hypothesis states that explicit claim-evidence governance will reduce unsupported behavioral conclusions while maintaining or improving behavioral discovery accuracy. Competing interventions include a methodology prompt or skill, a structured investigation protocol, externalized evidence records, and stronger mechanical enforcement. Their relative value remains an experimental question. Evaluation should use strong reverse-engineering baselines, identical tools and budgets, heterogeneous repositories, held-out behavioral checks, seeded or historical defects, repeated agent runs, and measures of unsupported claims, discovery accuracy, uncertainty calibration, reproducibility, sensitivity, cost, and downstream maintenance utility.

The resulting contribution is a testable theory of agentic evidence governance. Its success depends on measurable improvements over established practice and simpler prompting approaches. Its failure conditions are equally important: prompt-only parity, excessive abstention, poor cross-domain generalization, high operational cost, and equivalence with strong existing reverse-engineering workflows would narrow or reject the stronger framework claims.

Keywords: software reverse engineering; program comprehension; autonomous coding agents; behavioral evidence; characterization testing; test oracles; mutation testing; dynamic analysis; epistemic calibration; software maintenance

1. Research position and scope

Software maintenance begins with a knowledge problem. Engineers and agents must determine which behavioral claims about an existing system are actually supported before they can change that system safely. Source code exposes mechanisms and possible paths. Executions expose concrete outcomes under particular conditions. Documentation and history provide evidence about intent. Tests, formal properties, contracts, and domain judgments supply different forms of support for correctness. Each channel answers a different class of question.

The established academic home for this problem is software reverse engineering and program comprehension. Chikofsky and Cross (1990) defined reverse engineering around analysis of an existing system and the recovery of components, relationships, and higher-level representations. Program-comprehension research has examined how developers form and revise mental models, ask task-specific questions, and combine static and dynamic information during maintenance (von Mayrhauser and Vans 1995; Sillito, Murphy, and De Volder 2006; Cornelissen et al. 2009). Architecture reconstruction, feature location, dynamic analysis, specification mining, test-oracle research, and repository mining provide additional structured traditions within the same broad problem space.

The theory developed here sits inside that established landscape. Its proposed contribution concerns evidence governance for autonomous investigation. An agent can inspect large portions of a repository, execute code,

synthesize tests, generate fixtures, substitute dependencies, and narrate conclusions at a speed that exceeds ordinary human review. That capability makes the transition from observation to conclusion a first-class engineering problem. The central research object is therefore the relationship between behavioral claims and the evidence collected to support them.

The earlier concept of whole-codebase behavioral comprehension remains useful as a descriptive goal: understand consequential runtime behavior broadly enough to support maintenance, auditing, debugging, or change. The revised framing treats that goal as a reverse-engineering objective and focuses the proposed novelty on claim calibration, provenance, explicit evidence boundaries, measurement sensitivity, and agent-specific controls.

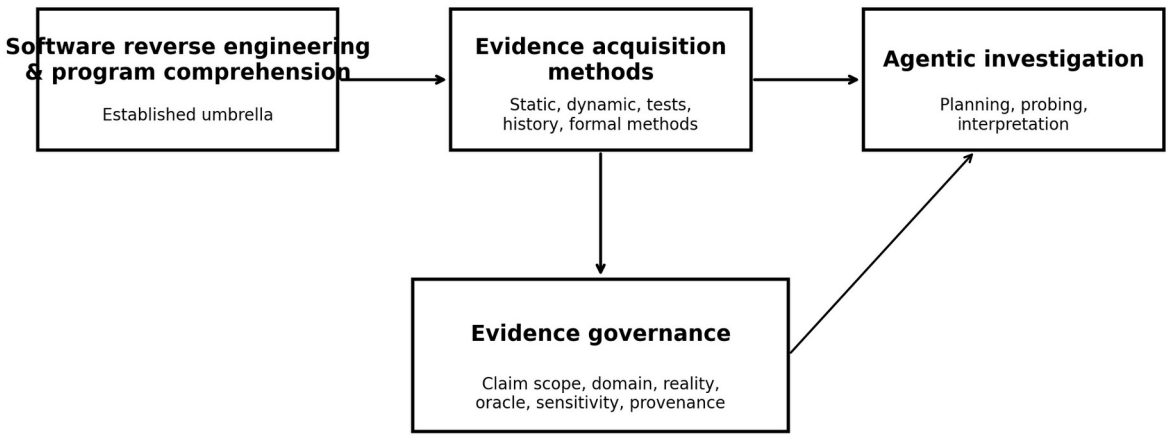


Figure 1. Intellectual positioning of the proposed research.

1.1 Research questions

1. Can explicit evidence governance reduce unsupported behavioral claims produced by autonomous reverse-engineering agents?
2. Which governance mechanisms create the effect: methodology instructions, structured evidence records, persistent state, mechanical gates, orchestration, or some combination?
3. How much behavioral discovery is preserved while unsupported claims decline?
4. Can a stable evidence model remain useful across materially different languages, architectures, test ecosystems, and execution environments?
5. Does the approach improve reproducibility and downstream maintenance outcomes enough to justify its operational cost?

2. Prior art and the appropriate novelty boundary

A rigorous novelty claim requires comparison with established reverse-engineering frameworks and techniques. Several existing traditions already provide structured processes for program understanding. Tilley's Reverse-Engineering Environment Framework (1998) categorizes support around cognitive models, reverse-engineering tasks, canonical activities, quality attributes, and environment characteristics. Software architecture reconstruction has developed process taxonomies and reference processes; Guamán et al. (2020), for example, synthesize prior work into the SIRA reference process and describe a process-level abstraction that is language-, paradigm-, tool-, application-, and domain-independent. These precedents demonstrate that general process abstractions within reverse engineering are established research territory.

2.1 Feathers: characterization and safe change

Michael Feathers' legacy-code work supplies a practitioner foundation for behavioral investigation. Characterization tests capture current behavior so that maintainers can detect change before restructuring poorly understood code. Seams create control over dependencies and execution conditions. The underlying objective is feedback: establish enough observability to reduce the risk of modification. The epistemic value of characterization is concrete continuity evidence. A characterization test says that a particular execution produced a particular result and can detect future deviation from that result.

The present synthesis extends the scope from safe local change to broader reverse-engineering claims. It combines characterization with structural analysis, differential and metamorphic relations, generated exploration, mutation-based sensitivity checks, environment fidelity, history, formal evidence, and explicit provenance. The extension is organizational: it asks how an autonomous investigator should combine these channels and how broadly each channel can support a behavioral conclusion.

2.2 Wilde and Scully: software reconnaissance

Wilde and Scully's software reconnaissance (1995) connects runtime execution with feature location. By comparing executions that exercise and omit a feature, the technique identifies source components associated with that feature. It is a focused reverse-engineering instrument for mapping observed functionality back to code. The present proposal uses reconnaissance as one possible evidence-acquisition method within a broader claim-calibration process. A reconnaissance result can support propositions about feature-associated regions and guide further probes, while additional evidence is needed for claims about intended behavior, correctness, environment-dependent semantics, and unexecuted paths.

2.3 Reverse-engineering frameworks and architecture reconstruction

Architecture-reconstruction literature shows that systematic, semi-automated, cross-language reverse-engineering processes already exist. These methods combine source extraction, structural analysis, clustering, views, patterns, stakeholder knowledge, and reconstruction activities. Their primary object is architectural representation and decision support. Evidence-governed agentic reverse engineering targets a different layer of control: how an autonomous investigator records and scopes behavioral evidence while it uses any available structural or dynamic technique.

Prior work	Primary objective	Typical evidence	Relationship to this proposal
Feathers / characterization	Safe modification of legacy code	Observed current behavior, seams, regression sensors	Supplies continuity evidence and feedback-loop practice
Wilde & Scully / reconnaissance	Locate code associated with features	Comparative execution coverage	Supplies a dynamic feature-location technique
Program comprehension	Build task-relevant understanding	Static, dynamic, textual, historical information	Supplies the broader cognitive and maintenance context
Architecture reconstruction	Recover architectural structures and views	Source facts, dependencies, clustering, stakeholder knowledge	Supplies structured reverse-engineering processes and representations
Testing research	Exercise behavior and detect anomalies or faults	Tests, generators, oracles, coverage, mutations	Supplies many evidence-generation and sensitivity methods
Proposed agentic layer	Calibrate claims produced by autonomous investigation	Linked claims, observations, domains, reality, oracles, sensitivity, provenance	Governs how heterogeneous evidence supports agent conclusions

Novelty boundary

The strongest defensible novelty claim concerns agent-specific evidence governance. The proposal earns research value only if explicit claim-evidence discipline improves outcomes beyond established reverse-engineering practice and strong methodology prompting.

3. The epistemic model of behavioral reverse engineering

Behavioral investigation becomes clearer when four objects are represented separately. The distinction prevents an agent from converting one kind of evidence into another by narrative convenience.

Object	Question	Strong evidence sources	Characteristic limitation
Mechanism	What structures can produce an effect?	Source, types, control/data flow, dependency graphs, static analysis	Possible mechanisms can remain dormant or unreachable in the relevant context
Actuality	What happened in a specified execution?	Tests, traces, replay, probes, field observations	Execution samples only the exercised domain and environment
Intent	What behavior is expected or historically required?	Specifications, contracts, documentation, issues, compatibility history, domain experts	Intent evidence can be stale, incomplete, or contradictory
Correctness	Does behavior satisfy the applicable criterion?	Oracles, formal specifications, invariants, domain rules, validated relations	Correctness depends on the adequacy of the governing criterion

A source path that appears to accept a null value establishes a mechanistic possibility. A test execution that preserves null establishes actuality for that test context. Documentation promising round-trip preservation supplies intent evidence. A validated schema or contract can provide a correctness criterion. Treating these statements as interchangeable produces systematic overclaiming.

3.1 Bounded evidence

Every behavioral observation is conditional. Its evidentiary scope depends on the program version, entry point, inputs, prior state, configuration, dependency versions, permissions, environment, timing, schedule, generated seed, and instrumentation. A durable behavioral record therefore needs enough contextual provenance to explain the conditions under which the observation was produced. The practical objective is explicit scope, because scope determines which later conclusions remain justified.

3.2 Evidence proportionality

Evidence proportionality links claim breadth to experimental breadth. A unit test can provide strong evidence for a pure local proposition. A mocked storage client can establish that an application issued a particular interaction against the substitute. A real database execution can establish database-specific semantics for the exercised setup. A staging result can support claims about the staging environment and shared production semantics. Each conclusion inherits the boundaries of the experiment that generated it.

3.3 Observation and verdict

Observations should remain stable records of what occurred. Verdicts should remain revisable interpretations. An observation can state that two serializers produced distinct logical values from the same input. A verdict can later classify that difference as a defect, compatibility behavior, intended format semantics, or unresolved anomaly. This separation preserves the ability to revise judgment as new requirements or historical evidence emerge.

4. Evidence channels and their limits

Whole-system understanding requires triangulation because each evidence channel exposes different information. The framework therefore treats evidence diversity as a design property of an investigation.

Evidence channel	Best-supported propositions	Main uncertainty
Structural / static	Possible mechanisms, reachability relations, data/control relationships, formally analyzed properties	Precision and soundness depend on the abstraction and trusted base
Controlled dynamic	Concrete outcomes for recorded inputs, states, configurations, schedules, versions, and environments	Unexercised behavior remains open
Relational	Disagreement, violated relations, cross-version deltas, round-trip failures	Comparability or the chosen relation can be wrong or incomplete
Historical / documentary	Intent, rationale, compatibility, incidents, prior expectations	Sources can drift from current system behavior
Real-boundary / field	Semantics of actual dependencies and operational contexts	Sampling, safety, availability, and environmental drift limit coverage
Sensitivity / mutation	Whether observation instruments react to selected plausible deviations	Results depend on the perturbation model and equivalence
Formal / semantic	Specified properties under explicit mathematical assumptions	Guarantees cover only the formalized property, model, and trusted implementation layers

4.1 Static and dynamic evidence reinforce each other

Static structure supplies mechanism and reachability hypotheses. Dynamic execution supplies concrete actuality. Joining them makes observations explainable and guides subsequent probes. A trace showing that one writer coerces null to an empty string gains investigative value when a structural analysis identifies every entry point reaching the same transformation. Conversely, a static call edge gains behavioral meaning when execution establishes that the edge participates in a consequential scenario.

4.2 Environment is part of the behavioral proposition

Configuration, infrastructure, dependency versions, identity, network behavior, clocks, and external services can materially determine program outcomes. Staging, containers, fakes, simulators, and mocks create different experimental realities. Their value depends on fidelity to the proposition under investigation. The framework records included and substituted reality explicitly so that later conclusions can preserve that distinction.

5. Established methods as evidence generators

The framework draws on established techniques and assigns each one a role in evidence acquisition, challenge, or interpretation. Selection should follow the behavioral question and available capabilities.

5.1 Characterization testing

Characterization tests record current behavior in executable form. They are especially valuable when change risk is high and normative specifications are incomplete. Their strongest proposition concerns continuity: a future execution can be compared with a previously observed result. The observation should retain relevant context so that contingent behavior is not promoted into an unconditional contract.

5.2 Property-based testing

Property-based testing expresses a relation or invariant and explores it across generated inputs. Its strength lies in broadening input exploration while retaining an interpretable behavioral rule. Its evidentiary quality depends on both the property and the generator. Round-trip preservation, idempotence, monotonicity, representation invariance, and domain invariants can provide strong relational oracles when they are justified by the domain.

5.3 Differential testing

Differential testing compares multiple implementations, versions, backends, paths, or interpreters on comparable inputs. Disagreement supplies an anomaly oracle when exact expected outputs are difficult to construct. The technique has produced substantial practical results, including compiler defects discovered through Csmith (Yang et al. 2011). The key governance question is comparability: the investigation must establish why the compared systems should agree on the property being examined.

5.4 Metamorphic testing

Metamorphic testing defines relations among multiple executions. It is useful when direct output oracles are weak. Examples include invariance under semantically irrelevant transformations, predictable scaling, and encode/decode relations. The metamorphic relation itself is part of the evidence model and must be justified independently of the implementation under test.

5.5 Combinatorial testing

Combinatorial interaction testing covers selected interactions among modeled parameters with far fewer cases than exhaustive enumeration. It is well suited to configuration-heavy systems. The exercised domain must preserve the parameter model and interaction strength because omitted dimensions remain outside the evidence.

5.6 Fuzzing

Fuzzing applies generated or mutated inputs and is highly effective when automatic failure oracles exist: crashes, sanitizer violations, assertion failures, hangs, protocol violations, or differential mismatches. Coverage-guided fuzzing can systematically expand exploration. Semantic defects that do not trigger an oracle remain invisible to the fuzzer, so the framework records which failure signals were actually observed.

5.7 Symbolic and concolic execution

Symbolic and concolic tools such as DART and KLEE use path constraints to generate executions that ordinary tests may miss. KLEE's evaluation on GNU Coreutils demonstrated high line coverage and discovery of serious defects (Cadar, Dunbar, and Engler 2008). Symbolic execution is especially useful as an exploration mechanism. Path explosion, environment modeling, solver limitations, and runtime support bound the claims that generated paths can support.

5.8 Mutation testing and fault injection

Mutation testing challenges the observation apparatus by introducing selected synthetic faults. Empirical work links mutation detection with real-fault detection more strongly than raw coverage alone in several studied systems (Just et al. 2014). Fault injection extends the same logic to service and infrastructure boundaries. The key question is whether tests and instrumentation react to plausible wrongness at the behaviorally relevant boundary.

5.9 Static analysis and formal methods

Static analysis can establish properties over classes of possible executions under a defined abstraction. Formal verification can provide stronger guarantees for explicitly modeled properties and trusted semantics. These methods complement dynamic evidence because they can reason about behavior beyond sampled executions. Their claims remain tied to the modeled property, assumptions, abstraction, and trusted base.

5.10 History and repository mining

Change history, incident history, blame data, issue discussions, and prior fixes provide intent and prioritization evidence. Process measures such as churn can identify areas with elevated change risk, while criticality and external exposure may make rarely changed code equally important. Historical evidence guides attention and explains compatibility constraints without substituting for current runtime observation.

6. Test suites as measurement systems

A test suite is a measurement system embedded in the repository. It selects inputs, constructs environments, invokes code, observes outputs or side effects, and applies oracles. This perspective changes the audit question from test quantity to evidentiary function: which behavioral propositions can the suite actually detect and distinguish?

6.1 Assertion quality

Execution breadth and observation quality are separate dimensions. An end-to-end test that asserts only an HTTP status can execute many components while observing little semantic behavior. A narrow deterministic test can provide strong evidence for a local invariant. The evaluation therefore examines what each test would notice if the relevant behavior changed.

6.2 Mocks, fakes, and substitutions

Substitution changes the experiment. A mock can isolate application logic and provide control over rare conditions. A real database can expose transaction, locking, type, collation, and persistence semantics. A local substitute can introduce its own mismatch with production. The relevant question is semantic materiality: which omitted behaviors could change the truth of the proposition the test is intended to support?

6.3 Redundancy and maintenance cost

Redundancy is evidentiary rather than textual. Similar tests may contribute independent value through different environments, entry points, contracts, or diagnostic power. Syntactically distinct tests can be behaviorally redundant. Consolidation decisions should therefore preserve independent fault-detection and contractual value while controlling runtime and maintenance burden.

6.4 Sensitivity of the apparatus

Passing tests demonstrate consistency with their current observations and oracles. Sensitivity experiments ask whether those observations are capable of detecting plausible deviations. Mutation, controlled breakage, and fault injection can reveal tests that execute a region without meaningfully observing its outputs. This second-order evaluation is central to the proposed framework because an autonomous agent can otherwise overvalue green test results.

7. Why autonomous agents change the reliability problem

Coding agents compress several roles that were historically separated among developers, testers, reviewers, tooling, and operators. One agent can read the implementation, infer expected behavior, generate tests, create mocks, run those tests, and summarize the result. That integration creates a distinctive circularity risk: evidence can inherit the same assumptions that generated the hypothesis.

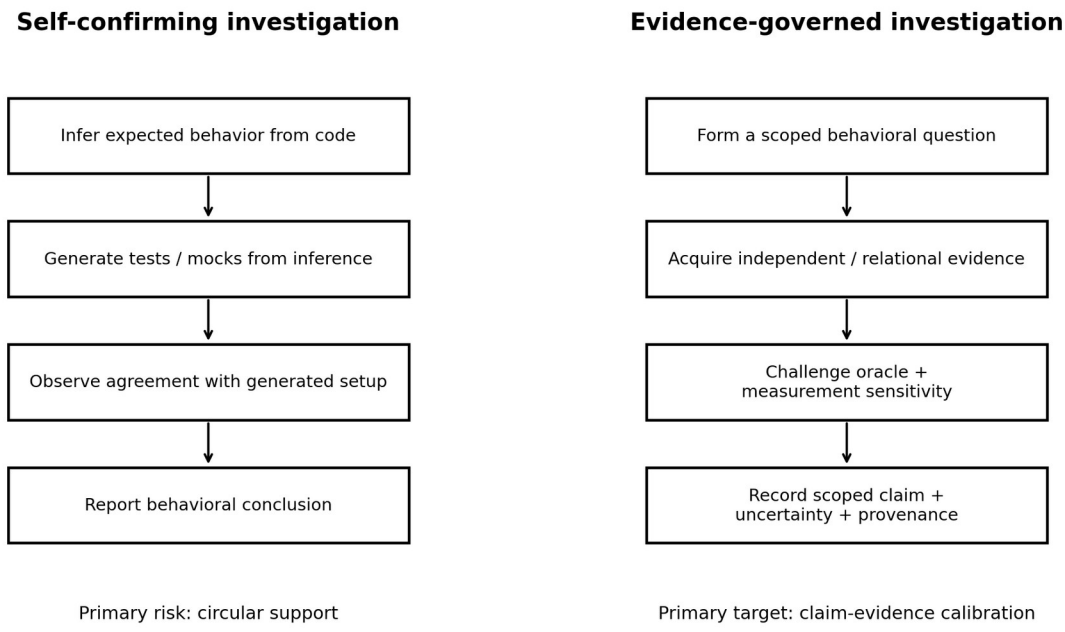


Figure 2. Agentic self-confirmation risk and an evidence-governed alternative.

7.1 Self-confirming tests and mocks

Consider an agent that reads a storage adapter, infers that the application should call an SDK method with a specific argument shape, creates a mocked SDK that accepts that shape, writes an assertion against the mock, and reports that persistence is verified. The resulting test can be internally consistent while excluding credentials, serialization acceptance, provider policy, network configuration, SDK behavior, and recovery semantics. The framework responds by recording the substitution and narrowing the supported proposition to the interaction that was actually exercised.

7.2 Fluent uncertainty collapse

Language models are optimized to produce coherent continuations and explanations. In reverse engineering, coherence can conceal unresolved intent or missing runtime evidence. A disciplined investigation therefore needs a stable representation for unresolved questions. `UNKNOWN` is useful when the available sources conflict, when behavior has not been exercised, or when an appropriate oracle is unavailable. The research question is whether explicit structures for uncertainty improve agent calibration without causing excessive abstention.

7.3 Scale, speed, and reproducibility

Agents can enumerate tests, generate cases, traverse repositories, and repeat experiments at large scale. That scale increases the value of machine-readable provenance because large quantities of generated evidence become difficult to audit from prose alone. Reproducible commands, inputs, versions, environments, and artifacts allow later investigators to replay the evidence without trusting the original explanation.

7.4 Existing evidence on agent-written tests

The source synthesis cites Hora and Robbes (2026), who analyzed agent-attributed commits across large TypeScript, JavaScript, and Python corpora and reported greater prevalence of test modifications and mocks in agent commits. That result is prevalence evidence. It motivates closer study of agent testing behavior while leaving causal questions about effectiveness open. The present proposal treats agent-generated tests as measurement artifacts whose quality should be evaluated directly.

8. A claim-evidence model for agentic investigation

The central representation associates every material behavioral conclusion with six dimensions:

Dimension	Meaning	Example question
P - Proposition	The behavioral statement being asserted	What exactly is the agent claiming?
D - Exercised domain	Inputs, states, configurations, schedules, and cases represented in the evidence	Which region of behavior was actually exercised?
R - Included reality	Real components and environments whose semantics were present	Which dependency and environment semantics were included or substituted?
O - Oracle	Criterion that makes an observation acceptable, anomalous, or incorrect	Why does this result count as evidence of success or failure?
S - Sensitivity	Evidence that the observation apparatus reacts to plausible perturbations	Would the tests notice relevant wrongness?
V - Provenance	Version, environment, tool, command, artifact, and reproduction context	Can another investigator reproduce the evidence?

A claim record can add status and uncertainty on top of this tuple. Useful statuses include supported, partially supported, contradicted, unsupported, and unknown. These labels should refer to a proposition at an explicit scope. They should never become global grades for the repository.

8.1 Example: persistence claim

Suppose an agent investigates the proposition `invoice objects round-trip through the persistence path without semantic loss`. A mocked repository test may support application-level call shape and local transformation behavior. A real PostgreSQL execution can add type, transaction, collation, and serialization evidence for the exercised schema and configuration. A round-trip oracle can compare the logical input and output. Mutation can test whether assertions detect selected field-loss transformations. Provenance records the database version, migration state, code revision, fixture, and invocation. The final claim can then state exactly which persistence path and data-shape classes were demonstrated and which operational contexts remain unresolved.

8.2 Evidence records and verdict records

A practical system should preserve evidence records separately from verdicts. Evidence records contain reproducible observations and context. Verdict records contain interpretation, intent, risk, or defect classification. This separation enables a later requirement, compatibility note, or production trace to revise the verdict while preserving the original observation.

9. Practical proto-implementation: an agent-facing investigation protocol

The lowest-cost implementation of the theory is a detailed methodology document supplied to an agent with repository and execution access. This prototype avoids premature infrastructure commitments and directly tests whether explicit investigative discipline changes agent behavior. The protocol developed alongside this paper asks the agent to preserve repository state, establish the unmodified baseline, inventory existing tests and substitutions, derive repository-specific behavioral surfaces, build a claim-evidence matrix, challenge important tests, generate additional evidence, improve the suite where warranted, and finish with an uncertainty ledger.

9.1 Why the protocol is a meaningful experiment

A protocol-only implementation isolates the methodological contribution. The same model can receive the same repository, tools, time budget, and execution permissions under two conditions: a normal comprehensive testing-review instruction and the evidence-governed protocol. Differences in unsupported claims, discovered behavioral

gaps, newly generated evidence, reproducibility, and cost can then be measured before building a dedicated runtime.

9.2 Existing tests remain first-class evidence

A realistic agent enters repositories that already contain verification infrastructure. The protocol therefore begins with the existing suite. The agent inventories tests, runs them before modification, identifies oracles and substitutions, examines skipped and flaky tests, maps tests to behavioral propositions, and decides which evidence should be retained, strengthened, replaced, consolidated, or quarantined. New tests are added when they provide needed evidence. The investigation can also use traces, probes, fuzzers, mutation tools, staging systems, symbolic executors, static analyzers, or historical sources when those methods better fit the question.

9.3 Behavioral surfaces emerge from the repository

The protocol avoids a universal checklist of software behaviors. Each repository yields its own consequential surfaces: serialization, state transitions, persistence, permissions, retries, browser behavior, configuration, concurrency, migration semantics, numeric precision, external providers, or other domain-specific behavior. The shared requirement is that the agent identify why a surface matters, which entry points expose it, which state and dependencies influence it, what evidence exists, and what uncertainty remains.

10. Universality and operational portability

A general theory needs a precise meaning of portability. The framework does not require one analysis engine to understand every language and domain. It requires the evidentiary concepts to remain useful while investigative mechanisms vary. Python, JVM, browser, native, mobile, distributed, and embedded systems can therefore use different test runners and analysis tools while producing comparable claim-evidence records.

10.1 Core stability as the portability criterion

Operational portability can be evaluated through core stability. A new ecosystem should primarily require new execution mechanisms, tool choices, or procedural guidance. Repeated changes to the meanings of proposition, exercised domain, included reality, oracle, sensitivity, provenance, observation, or uncertainty would indicate that the common abstraction is too weak or incorrectly designed. This gives universality a falsifiable architectural meaning.

10.2 Capability discovery

The proto-implementation can keep capability discovery informal: the agent examines the repository and identifies which build, test, tracing, mutation, fuzzing, symbolic, browser, database, or staging mechanisms are available. Later implementations could externalize that discovery. The research program should delay such infrastructure until protocol experiments show recurring failures that justify automation.

10.3 Portability metrics

- fraction of repositories where an unmodified baseline can be established;
- manual intervention required for setup and execution;
- ability to identify existing tests, mocks, external dependencies, and runtime boundaries;
- frequency with which the core evidence model requires repository-specific changes;
- quality of scoped conclusions when important capabilities are unavailable;
- cost of extending the method to a held-out language or application domain.

11. Competing implementation hypotheses

The theory does not determine a single implementation architecture. Several increasingly structured interventions should be treated as competing hypotheses. Their comparison is scientifically useful because a lightweight method may capture most of the benefit.

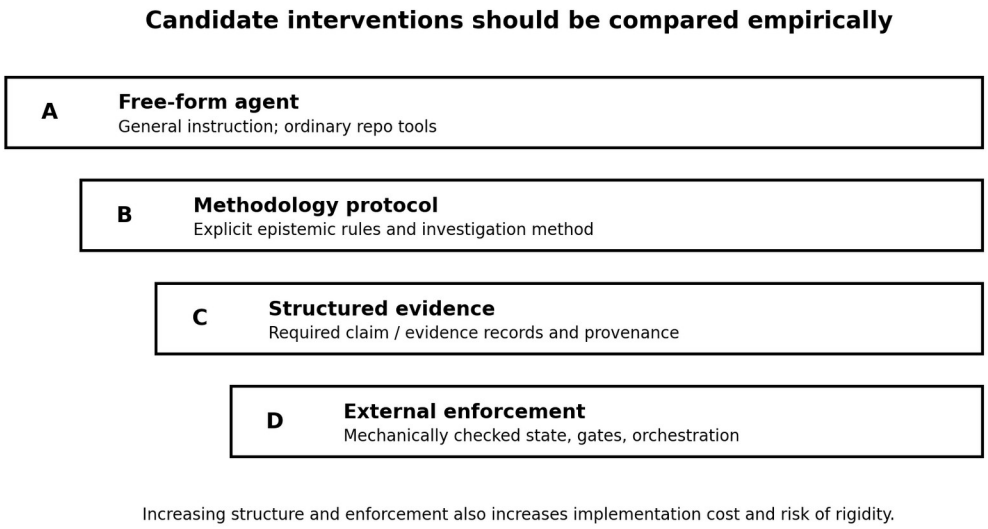


Figure 3. Candidate intervention ladder for empirical comparison.

Condition	Intervention	What it tests
A. Free-form agent	General reverse-engineering or test-review instruction with ordinary tools	Baseline capability of the model and tool environment
B. Methodology protocol	Explicit epistemic rules, investigation sequence, and uncertainty discipline	Whether guidance alone improves claim calibration
C. Structured evidence	Required evidence records, claim records, and persistent provenance	Whether externalized structure improves consistency and reproducibility
D. External enforcement	Mechanically checked states, gates, or orchestration for selected evidence properties	Whether enforcement adds value beyond guidance and structure

A strong result requires the more elaborate conditions to outperform simpler ones on meaningful outcomes after cost is considered. If Condition B matches Condition D, the practical contribution should remain a methodology protocol. If structured state or enforcement produces durable gains, the data can justify later implementation work.

12. Research hypotheses

The empirical program should begin with narrowly operationalized hypotheses. Each hypothesis must permit negative results and should be evaluated under identical tool access and comparable budgets.

H1 - Claim-evidence calibration

On unfamiliar-codebase behavioral investigations, agents using explicit claim-evidence governance will produce a lower rate of unsupported behavioral conclusions while maintaining or improving correct behavioral discovery.

H2 - Appropriate uncertainty

Evidence-governed agents will classify genuinely unresolved questions as unknown more accurately, while preserving useful conclusions when evidence is sufficient.

H3 - Reproducibility

An independent investigator will reproduce a larger fraction of material findings when the originating agent uses explicit provenance and evidence records.

H4 - Measurement sensitivity

Evidence-governed agents will identify more consequential weaknesses in existing tests and will create tests or probes that detect a larger fraction of relevant seeded or historical defects.

H5 - Operational portability

The core claim-evidence representation will remain stable across materially different repositories while the investigative techniques and tools vary.

H6 - Downstream maintenance utility

Teams or agents receiving persisted evidence-governed findings will spend less time rediscovering behavior and will make more accurate change-impact predictions on later maintenance tasks.

13. Experimental design

The evaluation must separate methodology from raw capability. Every condition should receive the same repository access, model family, context limits, terminal or coding tools, test permissions, network policy, and time or token budget wherever practical. Treatment agents should gain only the evidence-governance intervention being studied.

13.1 Repository sample

The repository set should include materially different conditions: strong and weak existing suites; unit-heavy and integration-heavy projects; libraries, CLIs, web applications, backend services, and stateful systems; multiple languages and build systems; external infrastructure; configuration-sensitive behavior; legacy code; mocks and fakes; and repositories with known historical failures. A held-out ecosystem should be introduced after the methodology is designed to test portability rather than memorized benchmark fit.

13.2 Task construction and ground truth

Tasks should ask consequential behavioral questions or require improvement of an existing verification system. Ground truth can come from seeded faults, historical bugs, independently authored executable specifications, authoritative contracts, hidden integration tests, system maintainers, validated dependency behavior, and held-out executions. Different tasks may require different truth sources. The benchmark should preserve ambiguity where intent is genuinely unavailable so that appropriate uncertainty can be scored.

13.3 Repeated runs

Agentic behavior is stochastic. Each repository-condition pair should therefore run multiple trials. Repeated runs allow measurement of variance in tool selection, discovered evidence, unsupported claims, and stopping behavior. A useful framework should improve the distribution of outcomes, not merely produce one persuasive demonstration.

13.4 Blinded scoring

Independent reviewers can evaluate behavioral claims without seeing which treatment produced them. Each claim can be scored for factual correctness, evidence support, scope calibration, appropriate uncertainty, and reproducibility. Blinding reduces the risk that reviewers reward the vocabulary of the framework simply because the framework defines the rubric.

14. Evaluation metrics

Metric	Operational question	Desired direction
Unsupported-claim rate	What fraction of substantive claims exceed their available evidence?	Lower
Behavioral discovery accuracy	How many consequential behavioral facts are correctly established?	Higher
False defect rate	How often are benign or intended behaviors labeled as defects?	Lower
Missed consequential behavior	How much important behavior remains undiscovered?	Lower
Uncertainty calibration	Does the agent use unknown when evidence is genuinely insufficient and commit when evidence is adequate?	Better calibrated
Reproducibility	Can an independent investigator recreate findings from recorded evidence?	Higher
Held-out behavioral validity	Do claims survive executions, states, or configurations not seen during investigation?	Higher
Measurement sensitivity	Do tests or probes detect relevant mutations, injected faults, or historical bugs?	Higher
Change-impact prediction	Can the investigator predict the behavioral effect of a later modification?	Higher
Operational cost	What time, tokens, tool calls, execution cost, and human intervention are required?	Lower for equal outcome
Run-to-run variance	How stable are conclusions and evidence quality across repeated trials?	Lower for core outcomes

14.1 Claim calibration as the primary outcome

Claim calibration directly tests the theory. A claim such as `persistence is correct` should score poorly when the evidence consists only of a mocked repository interaction. A scoped claim such as `the application issued the expected repository call for these exercised inputs against the configured substitute` should score well if the recorded execution supports it. The metric therefore rewards useful precision instead of rhetorical confidence.

14.2 Held-out execution

Held-out execution provides an objective challenge to generalization. After an agent completes an investigation, hidden inputs, states, configurations, or execution paths can test whether its behavioral claims survive beyond the observations that produced them. Claims with explicit scopes can remain correct even when broader hidden cases differ, which makes scope calibration measurable rather than purely stylistic.

15. Evidence that would weaken or falsify the proposal

A credible research program specifies outcomes that would reduce confidence in the framework. Several results would materially weaken the stronger claims.

- A strong methodology prompt or skill matches structured evidence and enforcement on claim calibration, discovery accuracy, reproducibility, and cost.
- Unsupported claims decline mainly because treatment agents abstain from making correct and useful conclusions.
- The evidence model requires frequent domain-specific reinterpretation, causing the supposed common core to change across repositories.

- Structured evidence adds substantial runtime, tool, or human-review cost with only small reliability gains.
- Experienced reverse-engineering workflows already achieve equivalent calibration and reproducibility without the proposed structures.
- Results disappear on held-out languages, architectures, or repository types.
- Independent reviewers cannot reliably agree on whether claims are supported, indicating weak construct validity for the primary metric.
- Agents learn to satisfy evidence forms mechanically while continuing to produce semantically unsupported conclusions.

These outcomes would still yield useful knowledge. They would identify which parts of the synthesis are methodological restatements, which parts are agent-specific, and which implementation mechanisms create unnecessary complexity.

16. Threats to validity and unresolved theoretical questions

16.1 Literature completeness

The current synthesis spans many established research areas but is integrative rather than exhaustive. A stronger novelty assessment should systematically search reverse-engineering environments, architecture reconstruction, software assurance cases, provenance systems, verification and validation, automated debugging, agentic coding, scientific workflow systems, and evidence-based safety engineering. Similar claim-evidence structures may already exist under different terminology.

16.2 Construct validity

The primary constructs - unsupported claim, evidence scope, included reality, oracle adequacy, sensitivity, and appropriate uncertainty - need operational definitions with acceptable inter-rater reliability. A theory of evidence governance becomes difficult to test if expert reviewers cannot distinguish supported from unsupported conclusions consistently.

16.3 Semantic gates

Mechanical enforcement is easiest for procedural facts such as repository revision, executed command, artifact existence, and declared substitutions. High-value judgments such as materiality of a dependency, validity of a metamorphic relation, or adequacy of an oracle require semantic reasoning. The fraction of useful governance that can be externalized is itself an empirical question.

16.4 Benchmark bias

Benchmarks built around examples that motivated the framework can exaggerate treatment effects. Evaluation should include broad maintenance tasks and historical failures selected independently of the framework authors. Repository and model diversity should be large enough to expose dependence on specific tooling or coding conventions.

16.5 Cost and maintainability

Evidence capture, replay, mutation, environment setup, and structured reporting can create operational burden. The research should measure whether stronger calibration justifies that burden and whether the resulting evidence remains useful after the repository evolves.

17. Research agenda

The theory supports a staged program that escalates implementation complexity only when evidence justifies it.

1. Prior-art consolidation. Conduct a systematic search focused on evidence governance, reverse-engineering process frameworks, assurance cases, provenance, and agentic software engineering.

2. Construct validation. Define supported claim, scope, uncertainty, oracle quality, and sensitivity precisely enough for independent reviewers to score them reliably.
3. Protocol pilot. Compare a strong general testing/reverse-engineering prompt with the full behavioral investigation protocol on a small set of repositories using identical agents and tools.
4. Failure analysis. Identify recurring errors that survive the protocol: forgotten provenance, circular oracles, overgeneralization, inappropriate substitutions, excessive abstention, or investigation drift.
5. Ablation study. Add structured evidence records, persistent state, selected gates, or orchestration one mechanism at a time and measure marginal value.
6. Cross-domain portability study. Introduce held-out languages and application domains and measure core-schema stability, setup cost, and claim calibration.
7. Longitudinal maintenance study. Reuse accumulated evidence during later modifications and measure rediscovery cost, regression rate, diagnosis time, and change-impact prediction.

18. Implications for practical agentic coding

The immediate practical implication is a change in the unit of review. Agent-generated tests should be reviewed as evidence instruments tied to specific behavioral propositions. A reviewer can ask which reality the test includes, where the expected value came from, which conditions were exercised, whether plausible faults would be detected, and which conclusions remain outside the test's scope. This review is compatible with ordinary unit, integration, system, property-based, fuzzing, or formal practices.

18.1 Testing overhauls as empirical probes

A comprehensive test-suite overhaul provides a useful proto-implementation because it forces the agent to interact with existing evidence, create new evidence, and make claims about adequacy. The protocol can require baseline preservation, behavioral-surface discovery, claim-evidence mapping, mock and oracle audits, sensitivity checks, targeted generation of new tests or probes, and an explicit uncertainty ledger. The resulting artifacts make the agent's epistemic behavior observable for research.

18.2 Human review remains relevant

Domain intent, compatibility obligations, safety criteria, financial rules, privacy requirements, and user expectations often require external judgment. Evidence governance helps by preserving the distinction between what the software did and why that behavior should be considered acceptable. Human review can then focus on normative questions instead of reconstructing the entire execution history from prose.

19. Conclusion

Software reverse engineering already contains a rich set of methods for recovering structure and behavior from existing systems. The agentic era changes the operational profile of that work because autonomous systems can acquire evidence, generate tests, choose substitutions, and produce conclusions within one integrated loop. That integration makes claim calibration a concrete engineering and research problem.

Evidence-governed agentic reverse engineering organizes the problem around explicit behavioral propositions and six evidentiary dimensions: exercised domain, included reality, oracle, sensitivity, provenance, and the proposition itself. The framework also preserves the distinction among mechanism, actuality, intent, and correctness. These concepts provide a common vocabulary for evaluating tests, traces, static analyses, generated cases, real-boundary executions, historical sources, and formal evidence without requiring one universal analysis engine.

The immediate implementation should remain modest: a rigorous agent-facing investigation protocol can test whether the methodology improves outcomes before larger infrastructure is built. The empirical comparison should include strong prompting, structured evidence, and stronger enforcement as competing interventions. Success requires lower unsupported-claim rates, preserved or improved discovery, better uncertainty calibration,

reproducibility, sensitivity, and acceptable cost across heterogeneous repositories. Negative results should narrow the proposal accordingly.

The central research question is therefore precise: can explicit governance of evidence improve autonomous reverse-engineering decisions enough to justify its complexity? That question can be answered experimentally, and its answer determines whether the synthesis becomes a useful control discipline for agentic software engineering or remains a conceptual restatement of good reverse-engineering practice.

References

Ammons, G., Bodík, R., & Larus, J. R. (2002). Mining specifications. Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, 4-16. https://doi.org/10.1145/503272.503275

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). The oracle problem in software testing: A survey. IEEE Transactions on Software Engineering, 41(5), 507-525. https://doi.org/10.1109/TSE.2014.2372785

Bertolino, A., De Angelis, G., Miranda, B., & Tonella, P. (2023). In vivo test and rollback of Java applications as they are. Software Testing, Verification and Reliability, 33(7), e1857. https://doi.org/10.1002/stvr.1857

Binder, R. V. (1994). Design for testability in object-oriented systems. Communications of the ACM, 37(9), 87-101. https://doi.org/10.1145/182987.184077

Cadar, C., Dunbar, D., & Engler, D. (2008). KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. 8th USENIX Symposium on Operating Systems Design and Implementation, 209-224.

Chen, T. Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T. H., & Zhou, Z. Q. (2018). Metamorphic testing: A review of challenges and opportunities. ACM Computing Surveys, 51(1), Article 4. https://doi.org/10.1145/3143561

Chikofsky, E. J., & Cross, J. H. (1990). Reverse engineering and design recovery: A taxonomy. IEEE Software, 7(1), 13-17. https://doi.org/10.1109/52.43044

Claessen, K., & Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming, 268-279. https://doi.org/10.1145/351240.351266

Clarke, E., Grumberg, O., Jha, S., Lu, Y., & Veith, H. (2001). Progress on the state explosion problem in model checking. Informatics: 10 Years Back, 10 Years Ahead, LNCS 2000, 176-194. https://doi.org/10.1007/3-540-44577-3_12

Cornelissen, B., Zaidman, A., van Deursen, A., Moonen, L., & Koschke, R. (2009). A systematic survey of program comprehension through dynamic analysis. IEEE Transactions on Software Engineering, 35(5), 684-702. https://doi.org/10.1109/TSE.2009.28

Cousot, P., & Cousot, R. (1977). Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. Proceedings of POPL, 238-252. https://doi.org/10.1145/512950.512973

Dit, B., Revelle, M., Gethers, M., & Poshyvanyk, D. (2013). Feature location in source code: A taxonomy and survey. Journal of Software: Evolution and Process, 25(1), 53-95. https://doi.org/10.1002/smr.567

Ernst, M. D., Cockrell, J., Griswold, W. G., & Notkin, D. (2001). Dynamically discovering likely program invariants to support program evolution. IEEE Transactions on Software Engineering, 27(2), 99-123. https://doi.org/10.1109/32.908957

Feathers, M. (2002). Working Effectively With Legacy Code. Object Mentor working paper, revised April 9, 2002.

Feathers, M. (2021, March 15). Michael Feathers: Looking back at Working Effectively with Legacy Code. The InfoQ Podcast.

Godefroid, P., Klarlund, N., & Sen, K. (2005). DART: Directed automated random testing. Proceedings of PLDI, 213-223. https://doi.org/10.1145/1065010.1065036

Graves, T. L., Karr, A. F., Marron, J. S., & Siy, H. (2000). Predicting fault incidence using software change history. IEEE Transactions on Software Engineering, 26(7), 653-661. https://doi.org/10.1109/32.859533

Guamán, D., Pérez, J., Díaz, J., & Cuesta, C. E. (2020). Towards a reference process for software architecture reconstruction. IET Software, 14(6), 592-606. https://doi.org/10.1049/iet-sen.2019.0246

Hora, A., & Robbes, R. (2026). Are coding agents generating over-mocked tests? An empirical study. Proceedings of the 23rd International Conference on Mining Software Repositories, 335-346. https://doi.org/10.1145/3793302.3793362

Inozemtseva, L., & Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. Proceedings of ICSE, 435-445. https://doi.org/10.1145/2568225.2568271

Jia, Y., & Harman, M. (2011). An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering, 37(5), 649-678. https://doi.org/10.1109/TSE.2010.62

Just, R., Jalali, D., Inozemtseva, L., Ernst, M. D., Holmes, R., & Fraser, G. (2014). Are mutants a valid substitute for real faults in software testing? Proceedings of FSE, 654-665. https://doi.org/10.1145/2635868.2635929

Kuhn, D. R., Kacker, R. N., & Lei, Y. (2010). Practical Combinatorial Testing. NIST Special Publication 800-142. https://doi.org/10.6028/NIST.SP.800-142

Leroy, X. (2009). Formal verification of a realistic compiler. Communications of the ACM, 52(7), 107-115. https://doi.org/10.1145/1538788.1538814

Manès, V. J. M., Han, H., Han, C., Cha, S. K., Egele, M., Schwartz, E. J., & Woo, M. (2021). The art, science, and engineering of fuzzing: A survey. IEEE Transactions on Software Engineering, 47(11), 2312-2331. https://doi.org/10.1109/TSE.2019.2946563

Mårtensson, T., Ancher, G., & Ståhl, D. (2023). Test environments for large-scale software systems: An industrial study of intrinsic and extrinsic success factors. Software Testing, Verification and Reliability, 33(3), e1839. https://doi.org/10.1002/stvr.1839

McKeeman, W. M. (1998). Differential testing for software. Digital Technical Journal, 10(1), 100-107.

Nagappan, N., & Ball, T. (2005). Use of relative code churn measures to predict system defect density. Proceedings of ICSE, 284-292. https://doi.org/10.1145/1062455.1062514

Rothermel, G., Harrold, M. J., von Ronne, J., & Hong, C. (2002). Empirical studies of test-suite reduction. Software Testing, Verification and Reliability, 12(4), 219-249. https://doi.org/10.1002/stvr.256

Schuler, D., & Zeller, A. (2013). Covering and uncovering equivalent mutants. Software Testing, Verification and Reliability, 23(5), 353-374. https://doi.org/10.1002/stvr.1473

Sillito, J., Murphy, G. C., & De Volder, K. (2006). Questions programmers ask during software evolution tasks. Proceedings of FSE, 23-34. https://doi.org/10.1145/1181775.1181779

Spadini, D., Aniche, M., Bruntink, M., & Bacchelli, A. (2017). To mock or not to mock? An empirical study on mocking practices. Proceedings of MSR, 402-412. https://doi.org/10.1109/MSR.2017.61

Tilley, S. R. (1998). A Reverse-Engineering Environment Framework. CMU/SEI-98-TR-005. Software Engineering Institute, Carnegie Mellon University.

Trautsch, F., Herbold, S., & Grabowski, J. (2020). Are unit and integration test definitions still valid for modern Java projects? Journal of Systems and Software, 159, 110421. https://doi.org/10.1016/j.jss.2019.110421

von Mayrhauser, A., & Vans, A. M. (1995). Program comprehension during software maintenance and evolution. Computer, 28(8), 44-55. https://doi.org/10.1109/2.402076

Wilde, N., & Scully, M. C. (1995). Software reconnaissance: Mapping program features to code. Journal of Software Maintenance: Research and Practice, 7(1), 49-62. https://doi.org/10.1002/smr.4360070105

Yang, X., Chen, Y., Eide, E., & Regehr, J. (2011). Finding and understanding bugs in C compilers. Proceedings of PLDI, 283-294. https://doi.org/10.1145/1993498.1993532

Yin, Z., Ma, X., Zheng, J., Zhou, Y., Bairavasundaram, L. N., & Pasupathy, S. (2011). An empirical study on configuration errors in commercial and open source systems. Proceedings of SOSP, 159-172. https://doi.org/10.1145/2043556.2043572